One of the reasons for the scalability problem is the proliferation of data sources and data consumers. An obvious bottleneck emerges when one central team manages and owns data along its whole journey: from ingestion through transformation and harmonization to serving it to all potential users. Splitting the team along the data pipe does not help much either. When engineers working on data ingestion change anything, they need to inform the group responsible for transformation. Otherwise, the upstream systems may fail, or worse, will process the data incorrectly. Required close collaboration among the engineers leads to the tight coupling of all data-related systems.

The other problem arises from the monolithic nature of data platforms, such as warehouses and lakes. They often lack the diversity to reflect the reality encoded in data derived from sources and domain-specific structures. Moreover, enforced flattening of data structures reduces the ability to generate valuable insights from the collected data, as crucial domain-specific knowledge gets lost in these centralized platforms.

We could observe this problem in one of the projects we worked on. A car parts manufacturing company was buying data related to failures of various parts. Even though the provider had information on the part provenance (the model the part was installed in), the buyer had no data models allowing it to store this information. As a result, components were analyzed separately, hampering R&D’s attempts to understand the big picture better.

Two more interwoven factors exacerbate the problems described earlier. One is unclear data ownership structure; the other is the responsibility for data quality. Data traveling through specialized teams loses its connection to its business meaning. Developers of centralized data processing systems and applications can’t and won’t fully understand the data content. In contrast, data quality cannot be assessed apart from its meaning.

Similar problems were recognized in other areas of software engineering and resulted in the emergence (and success!) of domain-driven design and microservices. Application of similar thinking (a focus on data ownership and shared tooling) to data engineering led to the development of the data mesh approach.